

Hierarchical Reinforcement Learning for Few-Shot Adaptation Against Non-Stationary Opponents

Conor Karl McKeag
University of Cape Town
Cape Town, South Africa
MCKCON007@myuct.ac.za

Gideon Sweijd
University of Cape Town
Cape Town, South Africa
SWJGID001@myuct.ac.za

Brad Scott
University of Cape Town
Cape Town, South Africa
SCTBRA008@myuct.ac.za

Abstract

This paper investigates the sample efficiency and adaptation speed of hierarchical reinforcement learning (HRL) methods against non-stationary opponents across three competitive game environments: μ RTS, Reinforce Tactics, and Go. We introduce the Strategic Latent-Hierarchy (SLH) agent, an HRL architecture that encodes opponent behaviour into a latent strategy vector to enable few-shot adaptation, and evaluate it against a flat PPO baseline in μ RTS. In Reinforce Tactics, we compare alternative Manager-Worker reward routing schemes under a controlled mid-training opponent strategy switch. In Go, we compare a Feudal Network agent against a flat PPO baseline, measuring recovery time following abrupt opponent strategy shifts of varying magnitude and frequency. Across all three environments, we expect HRL methods to outperform their flat RL counterparts in adaptation speed. The findings aim to contribute empirical evidence on the viability of HRL for competitive, non-stationary environments that are currently understudied in the broader literature.

1 Introduction

In standard reinforcement learning (RL), convergence to an optimal policy is guaranteed under the assumption that reward and transition distributions remain constant [23], yet a learning or changing opponent violates this by continuously altering the effective dynamics the agent has learned to exploit, rendering its policy suddenly ineffective against unseen or non-stationary opponents. To address such challenges, deep reinforcement learning (DRL) – which combines deep neural networks with RL to enable agents to learn directly from high-dimensional inputs – has proven highly effective, as demonstrated by DeepMind’s AlphaStar [27] in StarCraft II and OpenAI’s Five [3] in Dota 2, both of which involved non-stationary opponents capable of employing unusual strategies.

However, these works involved long training periods and large datasets in order to counteract the difficulties associated with these types of tasks. Additionally, these environments have long horizons and sparse delayed reward signals, a challenge that standard DRL methods struggle to address effectively [28].

Hierarchical reinforcement learning (HRL) addresses this by breaking down complex tasks into a hierarchy of sub-goals, allowing a high-level policy to plan abstractly over longer timescales, while a low-level policy learns to complete the individual sub-goals over a shorter timescale through intrinsic rewards [21].

Several breakthroughs have been made by integrating DRL methods into HRL, with early works such as h-DQN [14] and FuN [26] enabling HRL to operate in state spaces of higher dimensions than those of standard RL. However, more recent research such as HAC [15] has demonstrated improved sample efficiency and the ability

to learn hierarchical policies simultaneously in continuous control tasks with few rewards.

This project investigates the sample efficiency of HRL methods by evaluating and comparing several HRL approaches across three distinct environments featuring non-stationary opponents. The primary objective is to quantify the adaptation speed of these methods in response to non-stationary opponents and to benchmark their performance against standard flat RL baselines. To this end, this proposal first covers the necessary background knowledge for understanding HRL methods alongside a representative architectural example, before presenting the primary and secondary research questions with their motivations. The proposed methods and evaluation environments are then described, followed by a detailed breakdown of the planned experiments and their expected outcomes.

2 Related Work

This section first covers the necessary framework required for understanding HRL. Then an example of an HRL method is discussed briefly to highlight the advantages of HRL.

2.1 Hierarchical Reinforcement Learning Framework

HRL overcomes the standard challenges of complicated and long horizon tasks associated with reinforcement learning (RL) by breaking down complex tasks into multiple levels of sub-tasks. The hierarchy of sub-tasks is learned by a high-level policy operating at a low timescale, whereas the individual sub-tasks are learned by a lower-level policy operating at a faster timescale [9, 21]. The high-level policy decides which sub-task needs to be performed to complete the task optimally, and the low-level policy will then perform the primitive actions associated with completing the sub-task chosen by the high-level policy [9].

By performing this task decomposition, the length of the original task is reduced into shorter periods of actions through sub-tasks, effectively reducing the total horizon of the problem being solved. This difference in lengths is known as temporal abstraction and is a crucial component in HRL [24].

Crucially, in HRL, the high-level policy receives extrinsic rewards given by the environment, whilst the low-level policy receives intrinsic rewards, generated by the agent itself, for reaching the goal of a sub-task [14, 18]. For example, if the high-level policy sets a sub-goal of "reach the door", the low-level policy must receive an intrinsic reward regardless of whether the environment has given any reward. This aspect lends HRL to solving problems with sparse rewards [14].

This also allows HRL based methods to solve sequential decision-making tasks with significantly fewer interactions with the environment than flat, single policy RL, as demonstrated by Levy et al. [15] with the HAC framework, which required 3-5x fewer interactions to find good policies than flat RL methods [15, 19]. It has also been shown to outperform standard RL in tasks with long horizons and sparse rewards. [15, 18, 21, 26].

2.2 FeUdal Networks

FeUdal Networks (FuN) from Vezhnevets et al. [26] is a hierarchical framework inspired by feudal reinforcement learning [5], in which levels of hierarchy within an agent communicate to the level below it what must be achieved, but does not explicitly communicate how to do so. With each level operating at a different temporal resolution, specifically, the higher levels operate at a lower temporal resolution than the lower levels [5]. FuN expands this by using a fully differentiable neural network with two levels of hierarchy.

FuN consists of two levels, with the top level being a "Manager" and the bottom level being a "Worker". The Manager produces goals at a lower temporal resolution in a learnt latent state-space, while the Worker operates at a higher temporal resolution and produces primitive actions conditioned on external observation, its own state and the Managers goal. A high-level view of the hierarchy is shown in Figure 1.

Both the Manager and the Worker are implemented as Long Short-Term Memory (LSTM) [11], which are specialised Recurrent Neural Networks (RNN). The Manager's LSTM, however, is a novel RNN architecture introduced by Vezhnevets et al. [26] referred to as a dilated LSTM (dLSTM). The dLSTM allows the Manager to preserve memories for longer periods of time.

FuN makes use of abstract goals set by the Manager, which represent a direction in a learned latent state-space. The Manager is trained to predict these advantageous directions, whilst the Worker is trained to follow the goal direction by being intrinsically rewarded based on the cosine difference between the change in its state and the goal. Crucially, the Manager does not know what actions the Worker has taken and is only trained based on extrinsic rewards as a result of the Worker's actions. This approach ensures that the Manager does not make its decisions based entirely on managing the Worker and allows the Worker to focus on learning how to perform primitive actions based on the directional goal.

One of the advantages of FuN is its ability for transfer, through an ablative analysis Vezhnevets et al. [26] demonstrated that when a FuN agent trained on an ALE game is transferred to a different ALE game, it adapts quickly, often scoring higher than a fresh FuN agent.

3 Research Questions

This section will list the primary and secondary research questions that will be addressed by this project and will discuss the motivations related to the project.

3.1 Primary Research Question

How well do hierarchical reinforcement learning (HRL) methods achieve few-shot adaptation against non-stationary opponents in competitive video game environments, and is this a more effective method than existing flat RL methods?

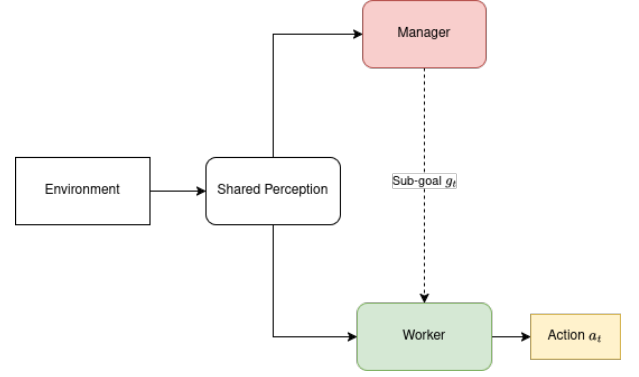


Figure 1: High-level representation of a FeUdal Network.

3.2 Secondary Research Questions

3.2.1 Research Question One (RQ1, Conor). Does the Strategic Latent-Hierarchy (SLH) agent adapt faster to unseen opponent strategies in μ RTS than TransformerNet [29]?

3.2.2 Motivation For RQ1. Real-time strategy (RTS) games provide a unique challenge for RL agents due to their large combinatorial spaces and sparse rewards [12]. Additionally, opponents in RTS games are often diverse, which presents the problem of non-stationarity as well. Hierarchical reinforcement learning has been shown to address these two issues of long-horizons and sparse rewards [21], however, little work has been done to evaluate HRL methods within these specific environments as well as evaluating HRL methods in the context of adapting to unseen strategies, which these environments provide. HRL has been shown to perform well when transferred to unseen tasks [26], learning quickly how to complete the task. However, this is purely from the view of completing an unrelated task, not from environment changes as a result of an opponent. Hence, more work should be done to explore whether HRL methods are able to adapt more quickly to unseen opponents than flat RL methods after learning some strategies.

3.2.3 Research Question Two (RQ2, Gideon). Do separated Manager and Worker reward signals in h-DQN slow win rate recovery after a sudden opponent strategy switch in a competitive turn-based game?

3.2.4 Motivation for RQ2. Competitive learning can induce non-stationarity [16, 17]. Goal-conditioned and feudal manager-worker methods [14, 18, 26] are, for the most part, trained and evaluated on long-horizon single agent, or cooperative control environments. By contrast, work on deep opponent modelling and meta-learned typically employs monolithic policies, that is, without a manager-worker hierarchy [1, 8, 17]. Reward routing determines which outcomes reinforce which level. When the opponent's behaviour regime changes, that assignment can sharpen or blur the learning signal each level receives during re-adaptation. This motivates a direct empirical comparison of alternative routing in *Reinforce Tactics* under a controlled mid-training strategy switch.

3.2.5 Research Question Three (RQ3, Brad). Does a feudal reinforcement learning improve recovery time following abrupt opponent strategy shifts, compared to flat RL approach (PPO)?

3.2.6 Motivation for RQ3. Dynamic shifting opponent strategies is a core challenge in competitive reinforcement learning [4, 10]. Flat architectures such as PPO [22] represent high-level goals and low-level actions within a single policy network, so any opponent-induced shift requires a complete network update that risks catastrophic forgetting of knowledge that remains relevant after the shift [10, 20]. Hierarchical architectures address this through temporal abstraction, allowing a high-level manager to revise strategic goals without overwriting the low-level workers who execute them [5, 24, 26]. However HRL methods have been validated majority on single-agent benchmarks [2, 24, 26], while adaptation against shifting opponents has largely been studied using flat policies [1, 17]. This motivates a direct comparison of hierarchical and flat approaches in Go under a controlled mid-game opponent strategy switch, using recovery time as the primary metric.

4 Methods

This section first covers the three environments that are being used for evaluation, then we will layout the three main methods for experiments, as well as the environment in which they are being evaluated.

4.1 Environments

4.1.1 μ RTS. Gym- μ RTS is an RL interface created by Huang et al. [12] for the RTS game μ RTS, an example of the environment is provided in Figure 2. It provides a simplified implementation of the standard challenges found in RTS games, such as combinatorial action space, real-time decision-making, partial observability, and stochasticity.

The environment was created with the computational costs of full-scale RTS games in mind, requiring only one GPU, three vCPU and 16GB RAM to train a state-of-the-art DRL agent, which was demonstrated with their modified Gridnet [7] and UAS agents.

There are numerous types of opponents available within μ RTS, sourced from annual tournaments. This makes it easy to test and train against specific strategies.

4.1.2 Reinforce Tactics. *Reinforce Tactics* is an open-source turn-based tactics game with a Gymnasium implementation for RL research [13]. Figure 3 shows a typical view. Agents read a grid of terrain, units, and resources, choose masked composite actions (recruitment, movement, and combat) and therefore face a combinatorial, long-horizon problem with structure capture, optional fog of war, and stochasticity from map and opponent choice, all in a light-weight multi-agent simulator.

The engine runs headless for training, supports editable and procedural maps for mixed pre-train and fixed test layouts, and offers scripted rule-based opponents of different strengths for controlled changes of opponent, alongside optional tournament and replay facilities for head-to-head comparison and off-line match review. This work uses *Reinforce Tactics* because MaskablePPO and feudal manager-worker training are already combined in one code path with direct hooks to opponent and map, so reward routing can be ablated with the other elements of the training and evaluation protocol held constant.

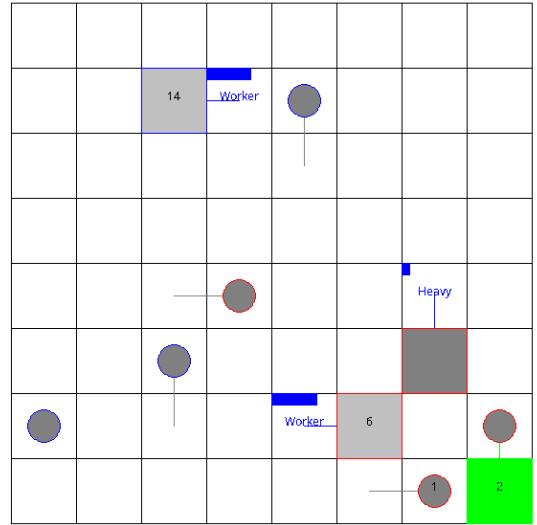


Figure 2: Screenshot from the μ RTS environment.



Figure 3: Screenshot from the *Reinforce Tactics* environment.

4.1.3 Go. The board game Go is used as the experimental environment for Research Question 3. Figure 4 shows a typical 19×19 Go board state during a training episode. Go is an ancient two-player board game in which both players always see the full board. Players alternate placing stones on intersections, with the winner determined by territory control at the end of the game using area scoring. Games average approximately 200 to 250 moves in total. The 19×19 board was chosen as it offers a trade-off between strategic complexity and computational cost, which provides a meaningful distinction between the architectural structures being tested.

Go was selected for three reasons. First, its sparse terminal rewards - a single +1 for a win or -1 for a loss received only at the

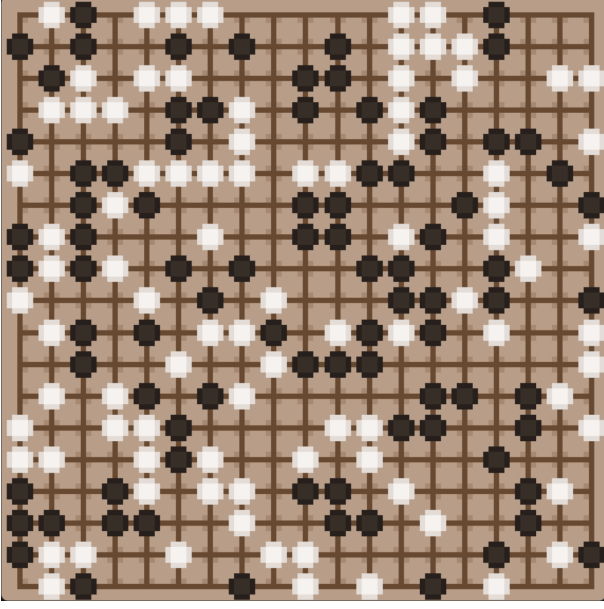


Figure 4: Screenshot of the 19×19 Go board environment.

final move with no feedback during the game — directly tests the hierarchical versus flat RL comparison, as the Worker in the Feudal Network can receive intrinsic rewards across these long stretches while the flat PPO agent must wait until the game ends [21, 26]. Second, Go demands both high-level positional planning and low-level tactical execution simultaneously, which connects onto the Manager - Worker structure of Feudal Networks [5, 26]. Third, Go is a perfect-information game, so any performance difference between the PPO baseline and the Feudal Network is purely architectural.

4.2 Strategic Latent-Hierarchy Agent Development

To address Research Question 1 (Section 3.2.1), we will develop a hierarchical architecture capable of long-term strategic planning, as well as being able to quickly adapt to unseen opponents. This will be evaluated in the μ RTS environment.

This architecture, the Strategic Latent-Hierarchy (SLH) agent, will consist of two layers: (1) a high-level manager policy operating at a lower temporal frequency which outputs a latent sub-goal vector, g_t , based on recent sequences of game state, and (2) a low-level worker policy operating at every game tick which takes the current game state and sub-goal from the manager policy as input.

Additionally, the manager receives an input from a Strategy Encoder. This encoder analyses the sequences that have occurred over the last N steps of the environment, summarising the opponent’s recent behaviour into a single fixed-length context vector referred to as the latent strategy vector, Z . A high-level view of the architecture is shown in Figure 5.

The high-level manager aims to produce strategic sub-goals based on the last N steps of game state which results in sequences that maximize the reward from the environment. These sub-goals are represented by a continuous latent sub-goal vector, g_t instead

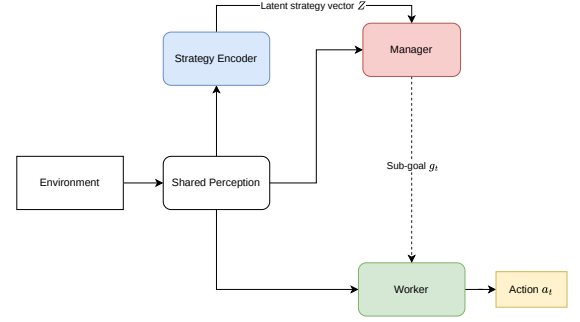


Figure 5: High-level diagram of the SLH Architecture

of discrete values. This allows the agent to learn more abstract representations of strategies in a similar fashion to that presented in the FeUdal Network (FuN) framework from Vezhnevets et al [26].

The low-level worker learns to produce actions which maximize an intrinsic reward received based on how closely it matches the given sub-goal before the next sub-goal is generated.

4.3 Feudal manager-worker training in Reinforce Tactics

To address Research Question 2 (Section 3.2.3), we train hierarchical agents in *Reinforce Tactics*¹: a Gymnasium-compatible, turn-based tactical game with discrete actions, imperfect information (fog of war), and scripted rule-based opponents at multiple skill levels [13]. All feudal conditions share the same map and observation interface; the independent variable is reward routing between Manager and Worker, following feudal separation of high-level credit from low-level sub-goal pursuit [5, 14] and softened Worker mixing as in FeUdal Networks [26].

A Manager policy issues a sub-goal every H environment timesteps, while a Worker policy selects primitive actions each timestep given the observation and active sub-goal. The Manager optimises segment returns from environment reward (win/loss shaping and intermediate signals as defined by the environment); the Worker optimises intrinsic sub-goal-completion reward and, in some conditions, a scaled extrinsic term. Training uses on-policy updates with separate actor-critic heads for Manager and Worker; the non-hierarchical baseline is flat Maskable PPO [22] with matched observation preprocessing where feasible.

4.4 Feudal Network Agent in Go

To address Research Question 3 (Section 3.2.5), a Feudal Network agent [5, 26] is trained and evaluated in the Go environment alongside a flat policy proximal optimization (PPO) baseline [22]. PPO uses a single shared network that maps the full (19, 19, 17) board observation directly to a probability distribution over all 362 actions, as it looks at both high-level positional strategy and low-level stone placement within one network (the 17 layers in the binary matrix capture the last 8 board states for black and white’s pieces, and 1

¹<https://github.com/kuds/reinforce-tactics>

layer of whose turn it is). Therefore any opponent-induced strategy shift forces an update across all policies, risking catastrophic forgetting of move-level skills that remain relevant after the shift has taken place [10, 20].

The Feudal Network separates this into two levels. The manager operates at a slower pace, setting a high-level strategic goal which is passed down to the worker as a direction to follow [5, 26]. The manager receives extrinsic rewards from the game outcome (+1 win, -1 loss). The worker operates at every individual timestep, selecting the stone placement or pass move that best fulfils the current manager goal, and receives an intrinsic reward based on how closely its actions match the given direction [26]. This intrinsic reward is critical in Go because the terminal reward only arrives after 200 to 250 moves, meaning the worker would receive no learning signal at all without it.

5 Experiments

The following section summarises the experiments which will be run in each environment. A detailed breakdown of each experiment is shown in Table 1.

5.1 μ RTS

In order to test Research Question 1 (Section 3.2.1), we train two agents: SLH agent and TransformerNet [29] as the baseline. They will both be initially trained against four seen opponents: Izanagi², Droplet³, NaiveMCTSAI and RandomBiasedAI⁴.

In order to correctly measure adaptation speed, the shaped rewards that each agent receives during training will be used as a means to compare. Specifically, we will take an average of the shaped reward received during training after 90M time steps to 100M time steps. Shaped rewards refer to the cumulative reward that the agent receives by completing certain tasks repeatedly in the environment.

5.1.1 Unseen Opponents. Following training, each agent will play against the following opponents: WorkerRushAI, LightRushAI, CoacAI⁵ and MixedBot⁶. During this, the shaped rewards that each agent receives against each opponent is tracked. Each agent will play up to 10M time steps, after which a quantitative evaluation is made.

5.1.2 Adaptation Speed Evaluation. We will use the tracking of the shaped reward metrics during training to compare to the shaped rewards received from playing against unseen opponents. Here we directly compare the SLH agent and TransformerNet to one another, whichever has a higher shaped reward within a shorter time frame is considered to adapt faster against each unseen opponent. For example, if the SLH agent had an average reward of 150 shaped during training and reached a value of 150 faster than TransformerNet against the same opponent, then the SLH agent is considered to adapt faster.

²<https://github.com/rubensolv/IzanagiBot>

³<https://github.com/zuozhiyang/Droplet>

⁴<https://github.com/Farama-Foundation/MicroRTS>

⁵<https://github.com/Farama-Foundation/MicroRTS>

⁶<https://github.com/AmoyZhp/MixedBotmRTS>

5.1.3 Ablation Study. An additional ablation study will be induced for this experiment. The SLH agent will have its strategy encoder removed, training will be repeated, and the adaptation speed evaluation will be run again. This is to determine whether the strategy encoder made a difference in the outcome of the experiment.

5.2 Reinforce Tactics

5.2.1 Reward-routing conditions. To address Research Question 2 (Section 3.2.3), we compare four agents (Experiments 9–16 in Table 1) that share the same observation, action masking, optimisation algorithm, opponent and map distributions, and environment-step budgets, and differ *only* in reward routing: (A) strict routing in the h-DQN style [14], where the Manager is trained on the summed segment environment return and the Worker is trained on intrinsic subgoal feedback alone (Experiments 9 and 13); (B) softened-Worker routing in the FuN style [26], where the Worker additionally receives a fixed fraction α of each step’s environment reward (e.g. $\alpha = 0.5$; Experiments 10 and 14); (C) informed-Manager routing, where the Manager additionally receives a fixed bonus β when its current subgoal is scored as achieved at segment end (a single 0/1 check on the game state) while the Worker is routed as in (A) (Experiments 11 and 15); and (D) a flat PPO baseline [22] with no Manager-Worker split (Experiments 12 and 16).

5.2.2 Evaluation protocol. Every condition first undergoes *stationary pretraining* (Experiments 9–12 in Table 1) against a fixed weak scripted opponent on diversified skirmish maps until the rolling win rate over the last $W = 50$ completed games (episodes, not timesteps) first reaches at least 60%. The opponent is then swapped at a halfway point in-game turn (turn-based game time, not raw environment-step index) to a stronger scripted policy on a single fixed evaluation map, so that the change of opponent is not confounded with terrain change. Mid-episode opponent changes are not part of the existing Reinforce Tactics API [13], so a simulator wrapper is built before switch data are collected. After the swap (Experiments 13–16), training continues on-policy under unchanged hyperparameters for the same additional environment-step budget T_2 for every condition, and we record the rolling win rate per completed game.

5.2.3 Metrics, compute. The independent variable is how many finished games pass after the opponent swap before the rolling win rate over the last W games is again within 10 percentage points of the rolling win rate measured just before the swap (same W as for the gate). We repeat every condition with at least three random seeds and report the average outcome plus a simple uncertainty band (standard error or bootstrap interval).

5.3 Go

This section describes the experimental design used to address Research Question 3. All experiments compare the PPO agent against the Feudal Network agent under identical conditions in the Petting-Zoo 19x19 Go environment [25].

5.3.1 External Baseline: GNU Go. GNU Go⁷ is used as an external reference opponent before Phase 1 training. It is an open-source,

⁷<https://www.gnu.org/software/gnugo/>

rule-based Go engine [6] that runs on CPU without GPU resources, plays at a consistent level on the full 19×19 board, and offers difficulty levels from 0 to 10, making it suitable for comparing the strength of agents trained against simple heuristic opponents.

5.3.2 Opponent Strategy Design. The white opponent plays using heuristic strategies implemented directly on the (19,19,17) board state matrix provided by PettingZoo⁸ [25]. Heuristic opponents are controlled over when and how strategy shifts are implemented into training, making clean recovery time measurements. Strategies include: random, aggressive, defensive, corner-focused, edge-focused, and greedy.

5.3.3 Phase 1 - Baseline Training. Both agents are trained separately against each white heuristic opponent for approximately 50M timesteps. At the end of each run, the average territory count over the final stable performance window is recorded as the baseline for Phase 2 recovery measurements.

5.3.4 Phase 2 - Abrupt Shift Experiments. Using Phase 1 baselines, both agents face controlled mid-training opponent strategy switches. Shifts are tested at three frequencies and three magnitude levels, giving 18 conditions in total. Previously seen strategies are reintroduced after the main conditions to evaluate the policy reuse metric [2, 24].

5.3.5 Evaluation Protocol. Recovery time is the primary metric: the number of completed Go games after a strategy shift before territory count returns to within 80% of the Phase 1 baseline. Secondary metrics are performance drop, adaptation speed, and policy reuse. All agents run on the same fixed 19×19 board ensuring post-shift scores are not influenced by environmental changes [10, 25].

6 Ethical Concerns

There are no ethical concerns related to this study. Any existing code used is open-source, and all training data used are generated procedurally through non-human opponents for each environment.

7 Expected Outcomes

This section discusses the main expected results of the project.

7.1 Overall Contribution

We expect that all trained HRL methods perform better than their flat RL counterparts. The environments in which the listed HRL methods are being applied are understudied in the broader research field. Thus, by performing the experiments listed, detailed studies are being contributed to the field within these complicated environments, as well as to the question whether HRL methods are viable for these tasks.

7.2 Strategic Latent-Hierarchy

We expect that the SLH agent will ultimately perform better than the flat RL methods and through the ablation study determine whether the strategic encoder is a necessary augmentation for HRL methods in these types of environments. Specifically, the SLH agent will be able to adapt faster than TransformerNet [29].

⁸<https://pettingzoo.farama.org/environments/classic/go/>

7.3 Alternative Reward Routing

The value of the results in Section 5.2.2 lies as much in interpretation as it does in observing clear differences in relative performance among (A)-(D). If after-switch recovery differs meaningfully across reward routings, this work could offer guidance on how to split Manager-Worker credit when non-stationarity is driven by an opponent strategy change. Conversely, if recovery says similar across routings, the same runs would still highlight how sensitive performance is to routing in this scenario. In both cases, this matters for deciding whether reward routing is a first-order design choice, and that the findings may support future tactical RL studies for which shifting opponents play a factor.

7.4 Reinforce Tactics

The opponent change is expected to depress rolling win-rate for all Agents (A-D) in the short run, then to recover toward the end of the W games played. We expect feudal conditions (A), (B) and (C) to recover faster on average than flat PPO (D), and we expect the different feudal reward routing variants to perform differently among each other, as the opponent shift should change which credit signal each level can use.

7.5 Go

We expect the Feudal Network agent to display faster recovery times than PPO following abrupt white strategy shifts in Go, with the advantage most showcased under major shift conditions where the opponent’s playstyle changes completely.

8 Project Plan

8.1 Resources

This section details the resources required by the project. All implementations will be done using Python, with PyTorch and Weights and Biases being the primary libraries used for implementation and experiment tracking, respectively. All experiments will be performed on the CHPC and Hex high-performance computers.

- TransformerNet Agent⁹
- Gym-μRTS Gymnasium Environment¹⁰
- Reinforce Tactics Gymnasium Environment
- Go PettingZoo Environment

8.2 Risks

The primary risks identified for this project are: (1) training may take too long, (2) implementation may be difficult due to inexperience, and (3) the choice of environment may result in limitations.

These primary risks and additional technical risks have been summarised in Table 2, as well as the mitigation strategies that will be in place.

8.3 Timeline and Milestones

The primary milestones highlighted in our timeline are grouped into categories: (1) Project Deliverables, (2) Architecture Development, (3) Training, (4) Tests & Experiments, and (5) Model Evaluation

⁹<https://github.com/NiklasZ/transformers-for-variable-action-envs>

¹⁰<https://github.com/Farama-Foundation/MicroRTS-Py>

& Metrics. In addition to these primary milestones, detailed sub-milestones and individual tasks have been highlighted in Figure 6.

References

- [1] Maruan Al-Shedivat, Trapit Bansal, Yura Burda, Ilya Sutskever, Igor Mordatch, and Pieter Abbeel. 2018. Continuous Adaptation via Meta-Learning in Nonstationary and Competitive Environments. In *Proceedings of the 6th International Conference on Learning Representations*.
- [2] Pierre-Luc Bacon, Jean Harb, and Doina Precup. 2017. The option-critic architecture. In *Proceedings of the AAAI conference on artificial intelligence*, Vol. 31.
- [3] OpenAI : Christopher Berner, Greg Brockman, Brooke Chan, Vicki Cheung, Przemyslaw Debiak, Christy Dennison, David Farhi, Quirin Fischer, Shariq Hashme, Christopher Hesse, Rafal Józefowicz, Scott Gray, Catherine Olsson, Jakub Pachocki, Michael Petrov, Henrique Pondé de Oliveira Pinto, Jonathan Raiman, Tim Salimans, Jeremy Schlatter, Jonas Schneider, Szymon Sidor, Ilya Sutskever, Jie Tang, Filip Wolski, and Susan Zhang. 2019. Dota 2 with Large Scale Deep Reinforcement Learning. *CoRR* abs/1912.06680 (2019).
- [4] Lucian Buşoniu, Robert Babuška, and Bart De Schutter. 2008. Multi-Agent Reinforcement Learning: An Overview. doi:10.1007/978-3-540-77922-9_1
- [5] Peter Dayan and Geoffrey E. Hinton. 1992. Feudal reinforcement learning. In *Proceedings of the 6th International Conference on Neural Information Processing Systems* (Denver, Colorado) (*NIPS'92*). Morgan Kaufmann Publishers Inc., San Francisco, CA, USA, 271–278.
- [6] GNU Go Contributors. 2009. GNU Go: A Free Program That Plays the Game of Go. <https://www.gnu.org/software/gnugo/>. Version 3.8. Free Software Foundation.
- [7] Lei Han, Peng Sun, Yali Du, Jiechao Xiong, Qing Wang, Xinghai Sun, Han Liu, and Tong Zhang. 2019. Grid-Wise Control for Multi-Agent Reinforcement Learning in Video Game AI. In *Proceedings of the 36th International Conference on Machine Learning (Proceedings of Machine Learning Research, Vol. 97)*, Kamalika Chaudhuri and Ruslan Salakhutdinov (Eds.). PMLR, 2576–2585. <https://proceedings.mlr.press/v97/han19a.html>
- [8] He He, Jordan Boyd-Graber, Kevin Kwok, and Hal Daumé III. 2016. Opponent Modeling in Deep Reinforcement Learning. In *Proceedings of the 33rd International Conference on Machine Learning*.
- [9] Bernhard Hengst. 2010. *Hierarchical Reinforcement Learning*. Springer US, Boston, MA, 495–502. doi:10.1007/978-0-387-30164-8_363
- [10] Pablo Hernandez-Leal, Michael Kaisers, Tim Baarslag, and Enrique Munoz de Cote. 2017. A Survey of Learning in Multiagent Environments: Dealing with Non-Stationarity. *CoRR* abs/1707.09183 (2017).
- [11] Sepp Hochreiter and Jürgen Schmidhuber. 1997. Long Short-Term Memory. *Neural Computation* 9, 8 (1997), 1735–1780. doi:10.1162/neco.1997.9.8.1735
- [12] Shengyi Huang, Santiago Ontanon, Chris Bamford, and Lukasz Grela. 2021. Gym-µRTS: Toward Affordable Full Game Real-time Strategy Games Research with Deep Reinforcement Learning. 1–8. doi:10.1109/CoGS2621.2021.9619076
- [13] Michael Kudlaty. 2025. *Reinforce Tactics: A turn-based strategy game for reinforcement learning research*. <https://github.com/kuds/reinforce-tactics>
- [14] Tejas D Kulkarni, Karthik Narasimhan, Ardavan Saeedi, and Josh Tenenbaum. 2016. Hierarchical Deep Reinforcement Learning: Integrating Temporal Abstraction and Intrinsic Motivation. In *Advances in Neural Information Processing Systems*, D. Lee, M. Sugiyama, U. Luxburg, I. Guyon, and R. Garnett (Eds.), Vol. 29. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2016/file/f442d33fa06832082290ad8544a8da27-Paper.pdf
- [15] Andrew Levy, Robert Platt, and Kate Saenko. 2019. Hierarchical Reinforcement Learning with Hindsight. In *International Conference on Learning Representations*. <https://openreview.net/forum?id=ryzECoAcY7>
- [16] Michael L. Littman. 1994. Markov Games as a Framework for Multi-Agent Reinforcement Learning. In *Proceedings of the 11th International Conference on Machine Learning*, 157–163.
- [17] Ryan Lowe, Yi Wu, Aviv Tamar, Jean Harb, Pieter Abbeel, and Igor Mordatch. 2017. Multi-Agent Actor-Critic for Mixed Cooperative-Competitive Environments. In *Advances in Neural Information Processing Systems*, Vol. 30.
- [18] Ofir Nachum, Shixiang (Shane) Gu, Honglak Lee, and Sergey Levine. 2018. Data-Efficient Hierarchical Reinforcement Learning. In *Advances in Neural Information Processing Systems*, S. Bengio, H. Wallach, H. Larochelle, K. Grauman, N. Cesa-Bianchi, and R. Garnett (Eds.), Vol. 31. Curran Associates, Inc. https://proceedings.neurips.cc/paper_files/paper/2018/file/e6384711491713d29bc63fc5eb5ba4f-Paper.pdf
- [19] Ofir Nachum, Haoran Tang, Xingyu Lu, Shixiang Gu, Honglak Lee, and Sergey Levine. 2019. Why Does Hierarchy (Sometimes) Work So Well in Reinforcement Learning? *CoRR* abs/1909.10618 (2019).
- [20] Charles Packer*, Katelyn Gao*, Jernej Kos, Philipp Krahenbuhl, Vladlen Koltun, and Dawn Song. 2019. Assessing Generalization in Deep Reinforcement Learning. <https://openreview.net/forum?id=rylKB3A9Fm>
- [21] Shubham Pateria, Budhitama Subagdja, Ah-hwee Tan, and Chai Quek. 2021. Hierarchical Reinforcement Learning: A Comprehensive Survey. *ACM Comput. Surv.* 54, 5, Article 109 (June 2021), 35 pages. doi:10.1145/3453160
- [22] John Schulman, Filip Wolski, Prafulla Dhariwal, Alec Radford, and Oleg Klimov. 2017. Proximal Policy Optimization Algorithms. *CoRR* abs/1707.06347 (2017).
- [23] Richard S. Sutton and Andrew G. Barto. 2018. *Reinforcement Learning: An Introduction* (second ed.). The MIT Press. <http://incompleteideas.net/book/the-book-2nd.html>
- [24] Richard S. Sutton, Doina Precup, and Satinder Singh. 1999. Between MDPs and semi-MDPs: a framework for temporal abstraction in reinforcement learning. *Artif. Intell.* 112, 1–2 (Aug. 1999), 181–211. doi:10.1016/S0004-3702(99)00052-1
- [25] Jordan Terry, Benjamin Black, Nathaniel Grammel, Mario Jayakumar, Ananth Hari, Ryan Sullivan, Luis S Santos, Clemens Dieffendahl, Caroline Horsch, Rodrigo Perez-Vicente, et al. 2021. Pettingzoo: Gym for multi-agent reinforcement learning. *Advances in Neural Information Processing Systems* 34 (2021), 15032–15043.
- [26] Alexander Sasha Vezhnevets, Simon Osindero, Tom Schaul, Nicolas Heess, Max Jaderberg, David Silver, and Koray Kavukcuoglu. 2017. FeUdal networks for hierarchical reinforcement learning. In *Proceedings of the 34th International Conference on Machine Learning - Volume 70* (Sydney, NSW, Australia) (*ICML'17*). JMLR.org, 3540–3549.
- [27] Oriol Vinyals, Igor Babuschkin, Wojciech M. Czarnecki, Michaël Mathieu, Andrew Dudzik, Junyoung Chung, David H. Choi, Richard Powell, Timo Ewalds, Petko Georgiev, Junhyuk Oh, Dan Horgan, Manuel Kroiss, Ivo Danihelka, Aja Huang, Laurent Sifre, Trevor Cai, John P. Agapiou, Max Jaderberg, Alexander S. Vezhnevets, Rémi Leblond, Tobias Pohlen, Valentin Dalibard, David Budden, Yury Sulsky, James Molloy, Tom L. Paine, Caglar Gulcehre, Ziyu Wang, Tobias Pfaff, Yuhuai Wu, Roman Ring, Dani Yogatama, Dario Wünsch, Katrina McKinney, Oliver Smith, Tom Schaul, Timothy Lillicrap, Koray Kavukcuoglu, Demis Hassabis, Chris Apps, and David Silver. 2019. Grandmaster level in StarCraft II using multi-agent reinforcement learning. *Nature* 575, 7782 (01 Nov 2019), 350–354. doi:10.1038/s41586-019-1724-z
- [28] Xu Wang, Sen Wang, Xingxing Liang, Dawei Zhao, Jincai Huang, Xin Xu, Bin Dai, and Qiguang Miao. 2024. Deep Reinforcement Learning: A Survey. *IEEE Transactions on Neural Networks and Learning Systems* 35, 4 (2024), 5064–5078. doi:10.1109/TNNLS.2022.3207346
- [29] Niklas Zwingenberger. 2023. Transformers as Policies for Variable Action Environments. *CoRR* abs/2301.03679 (2023). arXiv:2301.03679 doi:10.48550/ARXIV.2301.03679

A Experiments Tables

Experiment Number (Research Question(s) Addressed, Environment)	Experiment Name	Methodology	Evaluation Metric	Expected Outcome
1 (Question 1, μ RTS)	SLH adaptation speed evaluation to WorkerRushAI bot	Perform a second round of training on the trained SLH agent with only a single opponent for 10M steps. Then, observe the time it takes for the SLH agent to return to the average shaped reward value achieved during (90M - 100M steps) the initial training.	Highest shaped reward achieved at the end of the second round of training compared to the average shaped reward value achieved during the initial training.	The SLH agent will adapt quickly to the WorkerRushAI bot (5M - 10M steps).
2 (Question 1, μ RTS)	SLH adaptation speed evaluation to LightRushAI bot	Perform a second round of training on the trained SLH agent with only a single opponent for 10M steps. Then, observe the time it takes for the SLH agent to return to the average shaped reward value achieved during (90M - 100M steps) the initial training.	Highest shaped reward achieved at the end of the second round of training compared to the average shaped reward value achieved during the initial training.	The SLH agent will adapt very quickly to the LightRushAI bot (1M - 5M steps).
3 (Question 1, μ RTS)	SLH adaptation speed evaluation to CoacAI bot	Perform a second round of training on the trained SLH agent with only a single opponent for 10M steps. Then, observe the time it takes for the SLH agent to return to the average shaped reward value achieved during (90M - 100M steps) the initial training.	Highest shaped reward achieved at the end of the second round of training compared to the average shaped reward value achieved during the initial training.	The SLH agent will adapt quickly to the CoacAI bot (5M - 10M steps).
4 (Question 1, μ RTS)	SLH adaptation speed evaluation to MixedBot bot	Perform a second round of training on the trained SLH agent with only a single opponent for 10M steps. Then, observe the time it takes for the SLH agent to return to the average shaped reward value achieved during (90M - 100M steps) the initial training.	Highest shaped reward achieved at the end of the second round of training compared to the average shaped reward value achieved during the initial training.	The SLH agent will adapt slowly to the MixedBot bot (7M - 10M steps).
5 (Question 1, μ RTS)	TransformerNet adaptation speed evaluation to WorkerRushAI bot	Perform a second round of training on the trained TransformerNet agent with only a single opponent for 10M steps. Then, observe the time it takes for the TransformerNet agent to return to the average shaped reward value achieved during (90M - 100M steps) the initial training.	Highest shaped reward achieved at the end of the second round of training compared to the average shaped reward value achieved during the initial training.	The TransformerNet agent will adapt slowly to the WorkerRushAI bot (8M - 10M steps).
6 (Question 1, μ RTS)	TransformerNet adaptation speed evaluation to LightRushAI bot	Perform a second round of training on the trained TransformerNet agent with only a single opponent for 10M steps. Then, observe the time it takes for the TransformerNet agent to return to the average shaped reward value achieved during (90M - 100M steps) the initial training.	Highest shaped reward achieved at the end of the second round of training compared to the average shaped reward value achieved during the initial training.	The TransformerNet agent will adapt quickly to the LightRushAI bot (5M - 10M steps).

Experiment Number (Research Question(s) Addressed, Environment)	Experiment Name	Methodology	Evaluation Metric	Expected Outcome
7 (Question 1, μ RTS)	TransformerNet adaptation speed evaluation to CoacAI bot	Perform a second round of training on the trained TransformerNet agent with only a single opponent for 10M steps. Then, observe the time it takes for the TransformerNet agent to return to the average shaped reward value achieved during (90M - 100M steps) the initial training.	Highest shaped reward achieved at the end of the second round of training compared to the average shaped reward value achieved during the initial training.	The TransformerNet agent will adapt slowly to the CoacAI bot (9M - 10M steps).
8 (Question 1, μ RTS)	TransformerNet adaptation speed evaluation to MixedBot bot	Perform a second round of training on the trained TransformerNet agent with only a single opponent for 10M steps. Then, observe the time it takes for the TransformerNet agent to return to the average shaped reward value achieved during (90M - 100M steps) the initial training.	Highest shaped reward achieved at the end of the second round of training compared to the average shaped reward value achieved during the initial training.	The TransformerNet agent will adapt slowly to the MixedBot bot (9M - 10M steps).
9 (Question 2, Reinforce Tactics)	Phase 1 stationary pretraining of strict routing agent (A)	Train the strict-routing feudal agent (h-DQN-style separation [14]) against a fixed weak scripted opponent (SimpleBot) on diversified skirmish maps. The Manager is updated from summed segment environment return; the Worker is updated only from intrinsic subgoal feedback. Training stops once the rolling win rate over the last $W = 50$ completed games first reaches at least 60%.	Number of completed games to clear the 60% rolling win-rate gate, and the rolling win rate at the gate.	Strict routing reaches the gate within the pretraining budget but slower on average than the softened-Worker condition due to the absence of per-step extrinsic signal at the Worker.
10 (Question 2, Reinforce Tactics)	Phase 1 stationary pretraining of softened-Worker routing agent (B)	Train the FuN-style feudal agent [26] with identical opponent, map-diversification, and gate as Experiment 9. The Worker additionally receives a fixed fraction α (e.g. $\alpha = 0.5$) of each step's environment/extrinsic reward.	Number of completed games to clear the 60% rolling win-rate gate, and the rolling win rate at the gate.	The softened Worker condition reaches the gate fastest among the three feudal variants because tactical reward enters Worker updates directly.
11 (Question 2, Reinforce Tactics)	Phase 1 stationary pretraining of informed-Manager routing agent (C)	Train the informed-Manager feudal agent with identical opponent, map-diversification, and gate as Experiment 9. The Worker reward matches strict routing, while at segment end the Manager additionally receives a fixed bonus β if the current subgoal is achieved.	Number of completed games to clear the 60% rolling win-rate gate, and the rolling win rate at the gate.	The informed-Manager condition reaches the gate at a similar speed to agent (A).

Experiment Number (Research Question(s) Addressed, Environment)	Experiment Name	Methodology	Evaluation Metric	Expected Outcome
12 (Question 2, Reinforce Tactics)	Phase 1 stationary pretraining of flat PPO baseline (D)	Train a flat PPO agent [22] (no hierarchy) under identical environment, opponent, map-diversification, and gate as Experiment 9.	Number of completed games to clear the 60% rolling win-rate gate, and the rolling win rate at the gate.	The flat baseline reaches the gate within the pretraining budget and provides the non-hierarchical reference for the post-switch comparison.
13 (Question 2, Reinforce Tactics)	Phase 2 opponent strategy switch with strict routing agent (A)	Using the gated checkpoint from Experiment 9, switch the scripted opponent at a halfway in-game turn from SimpleBot to MediumBot on a single fixed evaluation map. Continue on-policy training with unchanged hyperparameters for the same additional environment-step budget T_2 as the other conditions, and record rolling win rate per completed game.	Recovery time: number of completed games after the switch before the rolling win rate over the last W games returns to within 10 percentage points of the pre-switch rolling win rate.	Strict routing recovers faster than agent (D) but slower than other feudal conditions.
14 (Question 2, Reinforce Tactics)	Phase 2 opponent strategy switch with softened-Worker routing agent (B)	Using the gated checkpoint from Experiment 10, apply the same opponent swap and post-switch budget T_2 as Experiment 13 under identical map and switching rule.	Recovery time as in Experiment 13.	Softened-Worker routing recovers fastest among the four conditions.
15 (Question 2, Reinforce Tactics)	Phase 2 opponent strategy switch with informed-Manager routing agent (C)	Using the gated checkpoint from Experiment 11, apply the same opponent swap and post-switch budget T_2 as Experiment 13 under identical map and switching rule.	Recovery time as in Experiment 13.	Informed-Manager routing recovers faster than strict routing through more stable high-level credit.
16 (Question 2, Reinforce Tactics)	Phase 2 opponent strategy switch with flat MaskablePPO baseline (D)	Using the gated checkpoint from Experiment 12, apply the same opponent swap and post-switch budget T_2 as Experiment 13 under identical map and switching rule.	Recovery time as in Experiment 13.	All feudal routing variants recover faster than the flat baseline.
17 (Question 3, Go)	PPO Phase 1 baseline training	Train the PPO agent against each of the six heuristic opponents (random, aggressive, defensive, corner-focused, edge-focused, greedy) for approximately 50M timesteps per opponent. Record the average territory count over the final stable window once performance has plateaued.	Average territory count over the final stable performance window per opponent.	PPO will achieve a stable territory count above the GNU Go baseline.

Experiment Number (Research Question(s) Addressed, Environment)	Experiment Name	Methodology	Evaluation Metric	Expected Outcome
18 (Question 3, Go)	Feudal Network Phase 1 baseline training	Train the Feudal Network agent against each of the six heuristic opponents under identical conditions and training budget as Experiment 9. Record the average territory count over the final stable window once performance has plateaued.	Average territory count over the final stable performance window per opponent.	The Feudal Network will achieve higher stable territory counts than PPO.
19 (Question 3, Go)	PPO Phase 2 low magnitude strategy shift	Using the PPO baseline from Experiment 9, introduce a low magnitude heuristic strategy switch at a pre-registered point. Continue training post-switch and record territory count per game until recovery. Repeated across three shift frequencies.	Recovery time: number of completed Go games before territory count returns to within 80% of the Phase 1 baseline. Secondary metrics: performance drop and adaptation speed.	PPO will recover slowly, requiring many games due to the risk of catastrophic forgetting across the entire shared policy network.
20 (Question 3, Go)	PPO Phase 2 medium magnitude strategy shift	Using the PPO baseline from Experiment 9, introduce a medium magnitude heuristic strategy switch at a pre-registered point. Continue training post-switch and record territory count per game until recovery. Repeated across three shift frequencies.	Recovery time: number of completed Go games before territory count returns to within 80% of the Phase 1 baseline. Secondary metrics: performance drop and adaptation speed.	PPO will display a larger performance drop and longer recovery time than under low magnitude shifts.
21 (Question 3, Go)	PPO Phase 2 high magnitude strategy shift	Using the PPO baseline from Experiment 9, introduce a high magnitude heuristic strategy switch at a pre-registered point. Continue training post-switch and record territory count per game until recovery. Repeated across three shift frequencies.	Recovery time: number of completed Go games before territory count returns to within 80% of the Phase 1 baseline. Secondary metrics: performance drop and adaptation speed.	PPO will exhibit the slowest recovery and largest performance drop.
22 (Question 3, Go)	Feudal Network Phase 2 low magnitude strategy shift	Using the Feudal Network baseline from Experiment 10, introduce a low magnitude heuristic strategy switch at a pre-registered point. Continue training post-switch and record territory count per game until recovery. Repeated across three shift frequencies.	Recovery time: number of completed Go games before territory count returns to within 80% of the Phase 1 baseline. Secondary metrics: performance drop and adaptation speed.	The Feudal Network will recover faster than PPO under low shifts.
23 (Question 3, Go)	Feudal Network Phase 2 medium magnitude strategy shift	Using the Feudal Network baseline from Experiment 10, introduce a medium magnitude heuristic strategy switch at a pre-registered point. Continue training post-switch and record territory count per game until recovery. Repeated across three shift frequencies.	Recovery time: number of completed Go games before territory count returns to within 80% of the Phase 1 baseline. Secondary metrics: performance drop and adaptation speed.	The Feudal Network will recover faster than PPO under medium shifts.

Experiment Number (Research Question(s) Addressed, Environment)	Experiment Name	Methodology	Evaluation Metric	Expected Outcome
24 (Question 3, Go)	Feudal Network Phase 2 high magnitude strategy shift	Using the Feudal Network baseline from Experiment 10, introduce a high magnitude heuristic strategy switch at a pre-registered point. Continue training post-switch and record territory count per game until recovery. Repeated across three shift frequencies.	Recovery time: number of completed Go games before territory count returns to within 80% of the Phase 1 baseline. Secondary metrics: performance drop and adaptation speed.	The Feudal Network will show the greatest recovery advantage over PPO under high magnitude shifts.

Table 1: Table of experiments

B Risk Matrix

Risk Description	Consequence	Probability	Impact	Mitigation Strategy
Team member drops out of honours	Zero consequences, none of the experiments are dependent on team members	Low	Low	
Insufficient computational resources	Experiments may not finish in time	Low	High	Ensure that proper usage of HPC resources is used.
Training duration exceeds time estimation	Agents may not be fully trained, thus the experiments will be delayed.	Medium	Medium	Start training as early as possible; correct training plans.
Implementation difficulties due to inexperience	Experiments may be delayed due to wasting time on poor implementation.	Medium	Medium	Proper planning and research into creating the agents.
Poor implementation affecting results	Poor implementation may lead to results being of low quality and/or unusable.	Low	Medium	Thoroughly test and validate implementations.
Difficulty capturing measurements	Lack of measurements to use for analysis.	Low	High	Determine how results can be acquired off of the HPC.
Data and/or project files lost	Complete loss of project.	Low	High	Ensure that git is being used and that backups are made consistently.
Choice of environment has limitations	Difficulty in measuring significant results.	Low	Medium	Preliminary testing of all environments.
Scope creep	Lack of time to analyse data/results due to scope creep.	Low	Medium	Stick to the planned scope and don't add more.

Table 2: Table of risks identified for this project.

C Project Schedule (Gantt Chart)

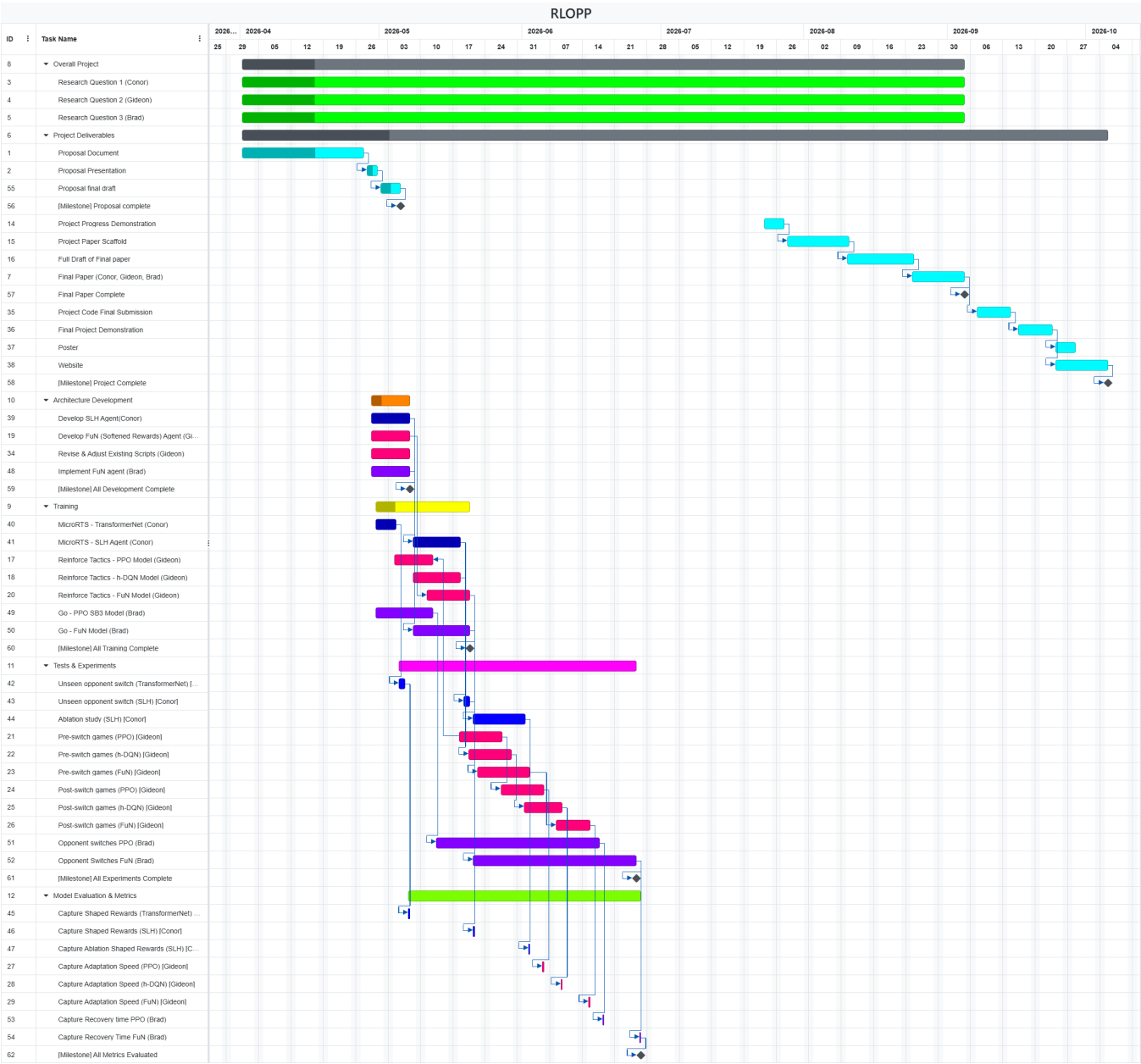


Figure 6: Provisional project Gantt chart.